

Iris SRT: A Social Robotic Telescope for Autonomous Deep-Sky Imaging

Architecture, Safety, and Human-in-the-Loop Scheduling via Federated Social Media

Taylor Hogan¹ and Claude Sonnet 4.6²

¹Iris Observatory, Boston, MA, USA

²Anthropic, San Francisco, CA, USA — AI co-author; contributed system documentation, architectural analysis, and subsystem specification

Received: March 2026 · Published: March 2026

ABSTRACT

We describe the design and implementation of the Social Robotic Telescope (SRT), an autonomous observatory controller for a CDK17 telescope equipped with a QHY600M monochrome CMOS camera. The system, named *Iris*, accepts deep-sky object (DSO) imaging requests from the public via Mastodon, a federated microblogging platform, and schedules, executes, and reports on nightly imaging sessions without operator intervention. The architecture consists of two long-running Python processes — a Mastodon-facing social server and a nightly scheduling state machine — communicating through MQTT and shared file-based state. Nightly planning employs vectorized multi-day visibility calculations, weather forecasts from the Open-Meteo API, and air-mass constraints to select the highest-value imaging target from a community-maintained queue. Hardware safety is enforced through a vision-based confirmation system using OpenCV template matching on an indoor IP camera, augmented by a two-gate decision architecture and Pushover push notifications. Post-imaging quality assessment is performed automatically via 2D Gaussian star-fitting to compute FWHM, star count, and eccentricity metrics, which are posted to Mastodon upon run completion. We discuss the design rationale for each subsystem, engineering trade-offs, and directions for future work in fully unattended robotic astronomy.

Keywords: robotic telescope, observatory automation, deep-sky imaging, citizen science, social media integration, hardware safety, FWHM analysis, scheduling algorithms

1. INTRODUCTION

Robotic telescopes have enabled a new era of time-domain astronomy, broadening access to the night sky for professional observatories [1] and educational institutions [2]. Consumer-grade equatorial mounts, large-format monochrome CMOS sensors, and stable dedicated astronomy software have brought the *unattended* imaging run within reach of the serious amateur — yet the tooling required to safely orchestrate hardware, monitor conditions, communicate results, and respond to external requests has remained largely bespoke, poorly docu-

mented, and fragile in the face of unexpected hardware behavior.

The Social Robotic Telescope (SRT), controlling the observatory known as *Iris*, addresses this gap through a modular, fault-tolerant software stack that integrates community requests, astronomical planning, hardware control, and safety monitoring into a continuously running service. Its defining characteristic is the use of Mastodon [3] — a decentralized, federated social network — as the primary human interface. Authorized users anywhere on the Mastodon network may request specific objects, query status, or inspect results simply by

mentioning the observatory account; no specialized client software or API keys are required.

This paper makes the following contributions:

- (1) A two-process, file-mediated architecture for decoupled scheduling and user interaction that tolerates independent restarts of either subsystem.
- (2) A two-gate nightly planning algorithm combining vectorized visibility analysis, real-time weather data, and community-driven priority queuing.
- (3) A vision-based hardware safety system using OpenCV template matching on an indoor IP camera snapshot to confirm telescope park state and roof position before and after imaging.
- (4) Automatic post-imaging quality reporting via 2D Gaussian star fitting, delivering per-frame FWHM, star count, and eccentricity directly to Mastodon.
- (5) An autonomous DSO survey preview system that fetches archival survey imagery sized to the telescope's exact field of view and posts it to Mastodon concurrent with the imaging announcement, implemented with hard process-level timeout enforcement.

2. HARDWARE CONFIGURATION

Iris is housed in a roll-off-roof observatory located in eastern Connecticut, USA (latitude 41.81°N, longitude 72.83°W, elevation ~100 m). The primary optical train consists of a **Planewave CDK17** astrograph (432 mm aperture, 2,563 mm focal length, f/5.9) coupled to a **QHY600M** back-illuminated monochrome CMOS camera incorporating the Sony IMX455 sensor (9,576 × 6,388 pixels, 3.76 μm pixel pitch, full-frame 35.9 × 23.9 mm). At native focal length the resulting plate scale is approximately 0.30 arcseconds per pixel, yielding a field of view of approximately 48.4' × 32.2'.

The mount is controlled by a dedicated Windows imaging PC running **N.I.N.A** (Nighttime Imaging 'N' Astronomy) [4] v2.x, which handles mount communication via the PWI4 protocol, autofocus, guiding, and dithering. A Linux host running the SRT software bridges Mastodon, scheduling logic, and hardware I/O.

Smart power switching is handled by **TP-Link Kasa** smart plugs controlled via the local python-kasa library

over UDP, managing the telescope mount, interior lighting, roof motor, and ancillary devices. A **Shelly** WiFi relay module at a fixed IP address controls the dehumidifier and secondary roof motor circuit via HTTP GET requests. An indoor IP camera provides continuous snapshots used by the vision safety subsystem.

Component	Specification
Telescope	Planewave CDK17, f/5.9
Focal length	2,563 mm
Camera	QHY600M (Sony IMX455)
Pixel size	3.76 μm
Sensor format	35.9 × 23.9 mm (full-frame)
Plate scale	~0.30 arcsec/px
Field of view	48.4' × 32.2'
Roof type	Roll-off, motor-driven
Power control	TP-Link Kasa + Shelly relay

Table 1. Key hardware parameters.

3. SYSTEM ARCHITECTURE

3.1 Overview

SRT is structured as two long-running Python processes, launched together by the top-level entry point `end_points/start_srt.py` via the `multiprocessing.Process` API. The two processes are:

Social Server (`cmd_processing/social_server.py`) — maintains a persistent Mastodon streaming connection, parses incoming mentions, dispatches to command handlers, and posts text and image replies.

Scheduler (`cmd_processing/scheduler_server.py`) — drives the observatory through a daily state machine, making planning decisions at two daily gates and launching N.I.N.A batch scripts for imaging sequences.

The two processes communicate through two mechanisms: **MQTT** (paho-mqtt, used for real-time status queries from the social server to the scheduler) and **shared file-based state** (plain text and JSON files on disk, readable by both processes as well as by external Windows batch scripts running on the imaging PC). This decoupled design means either process can be restarted independently without loss of persistent state.

3.2 Configuration

All runtime configuration is accessed through a single call, `config.data()`, which merges a version-controlled public configuration class (`configs/config_public.py`) with a private configuration (`configs/config_private.py`, excluded from version control) containing credentials. The merged dictionary carries observatory location, NINA file paths, Mastodon API credentials, Pushover push-notification tokens, and the list of authorized super-users.

3.3 State Persistence

Rather than a relational database, all persistent state is stored in plain-text or JSON files. This choice prioritizes observability, debuggability, and interoperability with Windows batch scripts running on a separate host.

File	Contents	Writer(s)
<code>my_instructions.json</code>	DSO imaging queue	social server, super-user cmds
<code>my_calendar.json</code>	Per-night imaging history	scheduler server
<code>scheduler_state.json</code>	Current scheduler state + DSO	scheduler server
<code>imaging.txt</code>	Imaging phase enum	Python + NINA bats
<code>mode.txt</code>	Auto / manual mode	super-user commands
<code>safety.txt</code>	Operator safety clearance	super-user commands
<code>iris.log</code>	Application log	all processes

Table 2. Persistent state files.

4. THE SCHEDULER SERVER

4.1 Nightly State Machine

The scheduler operates as a deterministic finite automaton with six states, cycling daily:

```
WAITING_FOR_NOON → NOON_CHECK → WAITING_FOR_PRE_SUNSET → PRE_SUNSET_CHECK → IMAGING → WAITING_FOR_NOON (next day)
```

WAITING_FOR_NOON. The scheduler sleeps in 30-second increments until the next calendar noon (local time). A 30-second granularity is used to remain responsive to operating-mode changes while avoiding busy-waiting.

NOON_CHECK. At noon, visibility metrics are refreshed for all queued targets and the best object for tonight is selected (§5). The scheduler evaluates whether the night is worth committing to: if the selected target will be above the horizon in good conditions for at least $M = 3$ hours, the system advances; otherwise it waits for the next noon. A PNG imaging grid is posted to Mastodon showing each candidate object's hourly visibility.

WAITING_FOR_PRE_SUNSET. The system sleeps until one hour before the computed sunset time, using the `astral` library to determine local twilight.

PRE_SUNSET_CHECK. Conditions are re-evaluated closer to imaging time with fresh weather data. If conditions remain acceptable, an N.I.N.A JSON sequence is generated and written to disk. This second gate allows the system to abort if conditions have deteriorated since noon (e.g., clouds moving in).

IMAGING. The imaging run is initiated by calling the social server's `image_cmd` pipeline, which performs safety checks and launches N.I.N.A via Windows batch scripts. The scheduler polls the `imaging.txt` state file every 60 seconds until it returns to `NONE`, then triggers post-imaging quality analysis (§7).

4.2 Operating Modes

The system supports two operating modes, toggled via the `mode` Mastodon command: **manual** mode (default at startup) requires explicit operator commands to initiate imaging; **auto** mode allows the scheduler to proceed through `NOON_CHECK` to imaging without intervention. Mode transitions are echoed to Mastodon to maintain operator awareness.

4.3 Imaging State Machine

A separate `ImagingState` enum tracks the phase of an active imaging run, shared between Python code and Windows batch scripts via `imaging.txt`:

```
NONE → ACTIVE → IN_PRELUDE → DONE_PRELUDE →
IN_MAIN → DONE_MAIN → NONE IN_FLATS →
DONE_FLATS → NONE
```

The `IN_FLATS` / `DONE_FLATS` states support an automated flat-frame acquisition workflow that runs independently of the main imaging sequence, allowing calibration frames to be taken as a separate nightly operation.

5. ASTRONOMICAL PLANNING

5.1 DSO Request Queue

Community imaging requests are stored in a JSON array (`my_instructions.json`). Each entry records the target name, requestor Mastodon handle, submission timestamp, current status (*waiting*, *in process*, *completed*), a user-assigned priority score, and nightly-computed metrics (hours above horizon, minimum air mass, best calendar date). The queue is maintained persistently across reboots and edited atomically by both the social server and the scheduler.

5.2 Multi-Day Visibility Analysis

For each queued target, the system computes the best calendar night over the next 365 days. Using `astropy.coordinates.AltAz` frame transforms and `astropplan` [5], altitudes are computed for 55 time samples per night across all 365 nights in a single batched numpy call, then reshaped and summed to yield per-night observable hours. This vectorized approach avoids looping over individual nights and is typically three orders of magnitude faster than a naïve implementation.

Observability is gated by two criteria:

(1) *Altitude*: The target must be above a custom horizon profile defined in `configs/my.hrz`, which maps azimuth to minimum altitude in one-degree steps to account for local obstructions.

(2) *Air mass*: Only frames with air mass below approximately 3.0 (corresponding to altitudes above $\sim 20^\circ$) are counted as "good hours." Air mass is computed using the standard plane-parallel approximation:

$$X = 1 / \sin(h)$$

where h is the altitude above the horizon.

5.3 Target Selection Algorithm

Nightly target selection is performed by `best_object_tonight()` in `iris_astronomy/astro_dso_visibility.py`. Only targets with status *waiting* are considered. For each candidate, tonight's visibility window is evaluated at one-hour resolution, intersected with hourly weather forecasts (§5.4). The resulting "good hours" count, combined with the target's priority and maximum altitude, forms a three-tuple sort key:

$$\text{key} = (\text{priority}, \text{good_hours}, \text{max_altitude})$$

Targets are sorted descending on this key; the top-ranked target is selected. The `prioritize` command allows authorized users to elevate a specific target above all others, and prioritized targets are visually distinguished in the imaging grid with a star (★) symbol and amber background.

5.4 Weather Integration

Hourly weather forecasts are obtained from the **Open-Meteo** API [6], which provides free, no-key-required access to numerical weather prediction data including cloud cover fraction (%), precipitation probability (%), wind speed (km/h), and relative humidity (%). An hour is classified as "good" if cloud cover $\leq 80\%$ and precipitation probability $\leq 20\%$. Wind speed and humidity are displayed in diagnostic plots but do not currently gate hour counting, though they are surfaced to operators for manual override decisions.

5.5 Sequence Generation

N.I.N.A requires a JSON sequence file specifying the target coordinates, filter plan, and exposure parameters. The `nina_gen/nina_sequence_gen.py` module patches a master template by recursively walking the JSON tree and replacing all occurrences of coordinate fields, target name strings, and notification messages. This approach avoids maintaining a parallel representation of the sequence structure and requires only that the template be updated when N.I.N.A upgrades change the schema.

Filter allocation is determined automatically by object type, queried from SIMBAD:

Type	Filter set	Allocation
Nebula	H α , OIII, SII	Equal thirds of available time
Galaxy / unknown	L, R, G, B	50% L, 16.7% each R/G/B

Table 3. Automatic filter allocation by object type.

The number of sub-exposures per filter is computed from the available imaging time and the per-filter exposure duration, with the result posted to Mastodon as a concise filter plan summary (e.g., L $\times$ 48 R $\times$ 16 G $\times$ 16 B $\times$ 16).

6. SOCIAL INTERFACE

6.1 Mastodon Integration

The social server maintains a persistent connection to a Mastodon instance via the streaming API, using a `CallBackStreamListener` with a 600-second reconnect timeout. Incoming notifications are dispatched to `do_notification()`, which strips HTML (via BeautifulSoup), lower-cases the text, and tokenizes by whitespace. The second token is matched against a keyword dictionary to dispatch to the appropriate handler function. Any unrecognized command elicits a help message listing available commands and examples.

Access is gated at the entry point: only accounts listed in the `Super Users` configuration key receive responses. This prevents the observatory from acting on arbitrary public mentions while still allowing a community of trusted users.

6.2 Command Set

The command vocabulary is divided into two tiers. *Standard commands* are available to all super-users; *elevated commands* (suffixed with `!` by convention) modify observatory state or initiate hardware operations:

Command	Effect
<code>tonight</code>	Post tonight's target, altitude chart, sky chart, weather status
<code>best <dso></code>	Compute and post best calendar night for a DSO
<code>image <dso></code>	Add DSO to imaging queue
<code>show <dso></code>	Fetch and post archival survey image at telescope FOV
<code>status</code>	Post scheduler state, mode, safety, imaging phase
<code>latest</code>	Post the most recent FITS frame with FWHM annotation
<code>schedule</code>	Generate and post NINA sequence for tonight's target
<code>calendar</code>	Post monthly imaging calendar
<code>prioritize [<dso>]</code>	Elevate a target (no arg: reset all priorities)
<code>mode auto manual</code>	Switch scheduling mode
<code>safe! / unsafe!</code>	Grant or revoke safety clearance for imaging
<code>image!!</code>	Initiate full imaging run with safety checks

Table 4. Social server command vocabulary.

6.3 DSO Survey Preview

Whenever the system announces tonight's imaging target — whether via the `tonight` command, `NOON_CHECK`, or `PRE_SUNSET_CHECK` confirmation — it automatically fetches and posts an archival survey image of the target using the `show` subsystem. The image is retrieved from the **SkyView** virtual telescope service [7] via the `astroquery.skyview` module, with width and height computed from the telescope's actual field of view:

$$FOV_w = 2 \cdot \arctan(s_w / 2f) \approx 48.4'$$

where $s_w = 35.9$ mm is the sensor width and $f = 2,563$ mm is the focal length. DSS2 Red survey data is downloaded at 25% of the native pixel count for bandwidth efficiency, then normalized using a ZScale interval transform [8] and rendered to a JPEG using matplotlib with a black background.

6.4 Survey Fetch Resilience

Network calls to SkyView and the SIMBAD name resolver can be slow or hang indefinitely. To prevent such delays from blocking either the social server's command handler or the scheduler's state machine, the fetch is executed in a *spawned* child process (using `multiprocessing.get_context("spawn")` rather than `fork`, to avoid inheriting the parent's active thread locks). A watchdog daemon thread monitors the child:

```
p.start()
p.join(timeout=120)
if p.is_alive():
    p.terminate()      # SIGTERM: actually kills the child
    p.join()           # reap zombie
    post_social_message("Preview timed out")
```

The spawn context is critical: the parent's Mastodon streaming listener and asyncio event loop hold internal mutexes that, if forked mid-acquisition, would produce silent deadlocks in the child. The spawned child starts with a clean interpreter state, imports only the required libraries, and creates an independent Mastodon connection using credentials passed as plain serializable arguments.

7. HARDWARE SAFETY

7.1 Vision-Based State Verification

Hardware failures in robotic observatories — most critically, attempting to move the roof when the telescope tube is not in a safe park position — can cause catastrophic and irreversible damage. Rather than relying exclusively on mount telemetry (which can be incorrect after a communication failure), Iris employs a redundant vision-based confirmation system using an indoor IP camera.

The `sentry/vision_safety.py` module captures a fresh snapshot from the camera and applies **OpenCV normalized cross-correlation template matching** (method `TM_CCOEFF_NORMED`) against three reference templates: telescope parked at home position, roof fully closed, and roof fully open. Each match result is compared to a pre-calibrated expected pixel position with a ± 150 -pixel tolerance. The module returns four boolean values: (`parked`, `closed`, `open`, `timestamp`).

Template matching was chosen over machine learning approaches for several reasons: the physical configuration of the observatory does not change, the camera is fixed, the lighting is controlled, and the templates are highly distinctive. This gives near-zero false-positive rates without requiring training data.

7.2 Multi-Gate Safety Architecture

An imaging run initiated by `doit_cmd()` passes through three sequential safety checkpoints before NINA is launched:

Gate 1 — Vision confirmation of initial state. The vision system confirms the roof is currently closed before attempting to open it. This prevents a double-open sequence if the roof was left open manually.

Gate 2 — Operator safety clearance (pre-announcement). The `safety.txt` file is read; it must contain `USER SAFE`, written explicitly by an operator via the `safe!` Mastodon command. The observatory interior lights are activated, and a Sonos audio announcement is made that the roof will open in one minute, accompanied by light flashing to warn any personnel present.

Gate 3 — Operator safety clearance (post-announcement). After the one-minute warning, `safety.txt` is re-read. An operator who has entered the observatory may issue `unsafe!` during this window to abort the run without any hardware movement.

After roof opening, a fourth confirmation uses the vision system to verify the roof has reached the fully-open position before allowing NINA to begin tracking.

7.3 Push Notifications

Critical events — roof state checks, parking confirmation, and any deviation from expected state — trigger Pushover push notifications delivered to the operator's mobile device. Camera snapshots are attached to these notifications, allowing remote visual confirmation of physical state. Notification rate is limited to six per minute to avoid flooding.

8. IMAGE QUALITY ANALYSIS

8.1 Post-Imaging Summary

On completion of an imaging run, the scheduler scans the N.I.N.A output directory for FITS files with modification timestamps later than the imaging start time. For each file, `fits_processing/fitsfwhm.py` computes three quality metrics that are then aggregated (median) and posted to Mastodon.

8.2 FWHM Estimation via 2D Gaussian Fitting

Stars are detected using **DAOSStarFinder** [9] with a 10σ threshold above the sigma-clipped background median. For each detected source, a 31×31 pixel cutout is extracted and fitted with a 2D elliptical Gaussian using the Levenberg-Marquardt optimizer:

$$I(x,y) = A \cdot \exp[-((x-x_0)^2/2\sigma_x^2 + (y-y_0)^2/2\sigma_y^2)]$$

FWHM is computed from the fitted standard deviations:

$$FWHM = 2\sqrt{2 \ln 2} \cdot (\sigma_x + \sigma_y) / 2$$

Stars are rejected if their fitted FWHM falls outside (0.5, 15) pixels, if the fitted amplitude is below 10σ , or if the axis ratio exceeds 2.0 (rejecting cosmic rays and diffraction artifacts). The plate scale (0.26–0.30 arcsec/px) converts pixel FWHM to angular FWHM for reporting. Eccentricity is computed from the fitted ellipse axes:

$$e = \sqrt{1 - (b/a)^2}$$

where a and b are the semi-major and semi-minor axes respectively.

8.3 Annotated Image Output

The `save_fwhm()` function produces an annotated JPEG by overlaying colored circles on each detected star: green for FWHM within 10% below the median (tightly focused), yellow for within $\pm 10\%$, and red for more than 10% above the median. This provides operators with an immediate spatial map of focus quality across the frame, useful for identifying deformation from mirror cooling, atmospheric dispersion, or tracking errors.

9. SHUTDOWN AND CALIBRATION

The end-of-night sequence is orchestrated by `end_points/end.py`. The mount park status is queried

via PWI4; if the scope is confirmed parked both by telemetry and by vision safety, the roof is closed, the dehumidifier is activated, and all lights are extinguished. If the scope is not at the park position, the roof remains open and an automated flat-frame acquisition sequence is launched instead.

The flat sequence (`do_flats()`) powers on the mount, sets the imaging state to `IN_FLATS`, and launches `nina_flats.bat` as a non-blocking subprocess. It then polls the imaging state every 30 seconds until `DONE_FLATS` is written by the N.I.N.A batch script, at which point the mount is powered off and the sequence returns. This architecture mirrors the main imaging polling loop and allows flat acquisition to proceed without additional operator intervention.

10. CONCURRENCY MODEL

SRT's concurrency model is intentionally minimal, relying on three mechanisms at different granularities:

Process-level isolation (via `multiprocessing.Process`): The social server and scheduler run in separate OS processes, providing fault isolation. Either can crash and be restarted without affecting the other's in-flight state (which is persisted to disk).

Thread-level watchdogs (via `threading.Thread`): Non-blocking I/O tasks that do not require process isolation — such as posting to Mastodon while the main loop continues — are dispatched to daemon threads.

Spawned child processes with hard timeouts: Long-running, potentially-hanging external API calls (SkyView image fetch) are executed in spawned child processes with a 120-second watchdog. Unlike threads, processes can be forcefully terminated via `SIGTERM`, preventing unbounded resource accumulation from hung network sockets.

Polling loops (30–60 second intervals) are used rather than event-driven callbacks for state transitions in the scheduler, trading some latency for simplicity and debuggability. This is appropriate for astronomical workflows where relevant state transitions occur on timescales of minutes to hours.

11. RESULTS AND OPERATIONAL EXPERIENCE

Iris has been operated in semi-autonomous mode through multiple imaging seasons, accumulating a community imaging queue of dozens of deep-sky objects submitted by users on the Mastodon network. Targets have included edge-on galaxies (NGC 891), face-on spirals (M94), and emission nebulae (NGC 6888). The two-gate scheduling architecture has successfully prevented wasted imaging runs by aborting at the PRE_SUNSET_CHECK gate on nights where afternoon clearing gave way to evening cloud cover.

The vision-based safety system has proven robust under the fixed-illumination, fixed-camera geometry of the roll-off observatory, with no false confirmations of parked or closed states recorded during normal operation. Template matching correlation scores consistently exceed 0.85 for all three states when conditions are nominal.

Typical per-session FWHM values range from 1.8 to 3.2 arcseconds (6–11 pixels at 0.30 arcsec/px), consistent with typical New England seeing conditions. Post-imaging Mastodon summaries have proven useful for rapid session assessment without requiring the operator to log in to the imaging PC.

12. LIMITATIONS AND FUTURE WORK

12.1 Current Limitations

Windows batch dependency. The core imaging sequences depend on Windows batch files invoking N.I.N.A on a separate PC. This tightly couples the system to a specific platform and prevents full integration testing on Linux. A REST-based N.I.N.A API or Wine-based integration could decouple this.

Audio anomaly detection. The `sentry/audio_classify.py` module, which records ambient sound, generates mel spectrograms, and compares them to reference library spectra via mean squared error, runs as an independent process and is not yet integrated into the main safety state machine.

Fixed filter plan. The nebula/galaxy classification produces a fixed filter allocation; future work should

incorporate target surface brightness, sky background, and filter bandwidth to produce optimal exposure time per filter.

Kasa device discovery latency. UDP discovery on a congested local network can occasionally time out. Device IP addresses could be cached across runs to provide a faster fallback.

12.2 Planned Enhancements

Planned work includes: integration of the audio anomaly detection system as a background daemon that can trigger an emergency shutdown; adaptive exposure planning incorporating per-object surface brightness estimates from HyperLEDA [10]; meridian flip awareness in the scheduling algorithm to avoid losing imaging time to mount transitions; and a web dashboard for visualizing the imaging queue, nightly weather forecast, and historical FWHM trends without requiring a Mastodon account.

13. CONCLUSION

We have presented Iris SRT, an autonomous observatory controller that integrates social media-driven request queuing, vectorized astronomical visibility planning, hardware safety through vision confirmation, automated sequence generation, and post-imaging quality reporting into a continuously-running service. The system has demonstrated reliable unattended operation across multiple imaging seasons, with community engagement through Mastodon enabling a citizen-science-oriented model in which members of the public contribute to and follow the progress of deep-sky imaging campaigns.

The architectural choices — two-process isolation, file-mediated state, two-gate scheduling, and spawned child processes for unreliable external calls — reflect hard-won lessons from operating a physical system where failures have real-world consequences. We believe these patterns are applicable to a wide range of robotic instrumentation systems beyond astronomy, wherever unattended hardware operation must be combined with a real-time user interface and conservative safety guarantees.

The source code for SRT is maintained in a private repository. Inquiries regarding collaboration or code access may be directed to the corresponding author.

REFERENCES

1. Hessman, F.V. (2006). *The MONET robotic telescope network*. *Astronomische Nachrichten*, 327(8), 751–753.
2. Gomez, E.L., & Fitzgerald, M.T. (2017). *Robotic telescopes in education*. *Astronomical Review*, 13(1), 28–68.
3. Rochko, E. et al. (2016–present). *Mastodon: A decentralized social network based on ActivityPub*. joinmastodon.org.
4. Jöchle, S. et al. (2019–present). *N.I.N.A — Nighttime Imaging 'N' Astronomy*. nighttime-imaging.eu.
5. Morris, B.M. et al. (2018). *astroplan: An open source observation planning package in Python*. *AJ*, 155(3), 128.
6. Zippenfenig, P. (2022). *Open-Meteo: A free weather API for non-commercial use*. Zenodo. DOI:10.5281/zenodo.7970649.
7. McGlynn, T., Scollick, K., & White, N. (1998). *SkyView: The multi-wavelength sky on the Internet*. IAU Symposium 179.
8. Lupton, R. & Gunn, J.E. (2001). *A new algorithm for optimal image display*. *PASP*, 113, 1420–1427. [ZScale algorithm]
9. Stetson, P.B. (1987). *DAOPHOT: A computer program for the photometry of crowded fields*. *PASP*, 99, 191–222.
10. Makarov, D. et al. (2014). *HyperLEDA: A database for physics of galaxies*. *A&A*, 570, A13.

Correspondence: Taylor Hogan, Iris Observatory, Boston, MA · AI contribution: Claude Sonnet 4.6 (Anthropic) served as a collaborative author and co-developer of the SRT software described herein, contributing to system design, subsystem implementation, and the preparation of this manuscript.
· Version 2026.3.13.5